Palm Targeting

1. Content Description

This course implements the acquisition of color images and the use of the MediaPipe framework to detect a hand and output its coordinates. This can be further integrated with a car chassis or robotic arm to track hand movements. This section requires inputting commands in the terminal. The appropriate terminal will be selected based on your board type. This course uses a Raspberry Pi 5 as an example.

For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this section in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] --[5.Enter Docker (For JETSON Nano and RPi 5)]**.

For Orin and RDK motherboards, simply open the terminal and enter the commands mentioned in this section.

2. Program Startup

For the Raspberry Pi 5 and jetson nano controller, you must first enter the Docker container. For the Orin and RDK controller, this is not necessary.

Entering the Docker container (see the Docker course for steps: 4. Docker Startup Script).

All commands below must be executed within the same Docker container.**(See the Docker course for steps: 3. Docker Commit and Multi-Terminal Entry).**

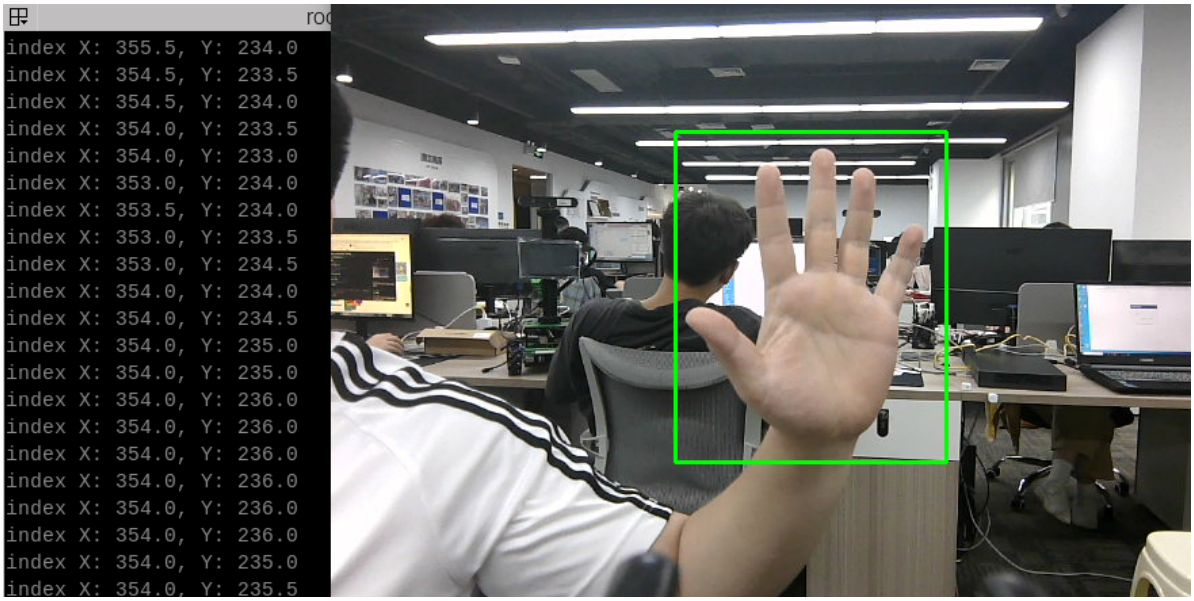
First, in the terminal, enter the following command to start the camera.

```
#usb camera
ros2 launch usb_cam camera.launch.py
#nuwa camera
ros2 launch ascamera hp60c.launch.py
```

After successfully starting the camera, open another terminal and enter the following command to start the palm tracking program.

```
ros2 run yahboomcar_mediapipe 11_FindHand
```

After starting the program, as shown in the image below, when a hand is detected, it will be outlined in green on the screen, and the center coordinates of the hand will be output in the terminal.



3. Core Code Analysis

Program Code Path:

- Raspberry Pi 5 and Jetson Nano board

The program code is running in Docker. The path in Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/11_FindHand.py`

- Orin board

The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/11_FindHand.py`

- RDK board

The program code path is `/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/11_FindHand.py`

Import the necessary library files.

```
import threading
import numpy as np
import time
import os
import sys
import cv2 as cv
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import Bool
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
sys.path.append('/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe')
from media_library import *
```

Initialize data and define publishers and subscribers,

```
ef __init__(self):
    super().__init__('hand_ctrl_arm_node')
    #Call the media_library library to create an object of the HandDetector
class
    self.hand_detector = HandDetector()
    #create a publisher
    self.bridge = CvBridge()
    #Define subscribers for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type ==
'nuwa' else '/usb_cam/image_raw'

    self.subscription = self.create_subscription(
        Image,
        topic_name,
        self.image_callback,
        10)
```

Color image callback function,

```
def image_callback(self, msg):
    #Use CvBridge to convert color image message data into image data
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    #Pass the obtained image into the process function for palm detection
    frame = self.process(frame)
    cv.imshow('frame', frame)
```

`process` function,

```
def process(self, frame):
    # Call the object method to perform palm detection and return the detected
image as well as the lmList list and bbox list
    frame, lmList, bbox = self.hand_detector.findHands(frame)
    # If the lmList list is not empty, it means that a palm has been detected.
    if len(lmList) != 0:
        threading.Thread(target=self.find_hand_threading, args=(lmList,
bbox)).start()
    return frame
```

`find_hand_threading` function,

```
def find_hand_threading(self, lmList, bbox):
    fingers = self.hand_detector.fingersUp(lmList)
    angle = self.hand_detector.ThumbTOforefinger(lmList)
    value = np.interp(angle, [0, 70], [185, 20])
    # Get the xy coordinates of the palm. bbox stores the xy coordinates of the
upper left and lower right corners of the palm
    indexX = (bbox[0] + bbox[2]) / 2
    indexY = (bbox[1] + bbox[3]) / 2
    print("index X: %.1f, Y: %.1f" % (indexX, indexY))
```

The definition of the Medipipe recognition class can be found in the media_library library, located in the yahboomcar_mediapipe package.

- Raspberry Pi 5 and Jetson Nano board

The path in Docker is `/root/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/media_library.py`

- Orin board

The program code path is `/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/media_library.py`

- RDK X5 board

The program code path is `/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/media_library.py`

In this library, we leverage the native mediapipe library to extend and define multiple classes. Each class defines different functions.

We simply pass in parameters as needed. For example, the HandDetector class defines the following functions:

- findHands: Finds hands
- fingersUp: Fingers extended upward
- ThumbTOforefinger: Detects the angle between the thumb and index finger
- get_gesture: Detects gestures